

Documentación Técnica

Gestión de Oficinos de Embargo y Enrutamiento de Casos

Solución de automatización en n8n con agente LLM e integración MCP

David Palacio Rubiano

Prueba técnica — Ingeniero de Software
Junio 2026

1. Resumen de la solución

Se construyó en n8n una solución funcional que recibe oficios judiciales de embargo, valida su consistencia mínima, extrae y normaliza sus datos, clasifica el caso hacia la plataforma adecuada, asigna una prioridad, genera un resumen ejecutivo mediante un agente LLM, produce un JSON estructurado de salida, emite un acuse de recibo al solicitante y registra la trazabilidad de la ejecución.

La solución se entrega en dos versiones complementarias:

- **Flujo base:** versión autónoma que ejecuta toda la lógica con nodos nativos de n8n, sin dependencias externas.
- **Flujo con MCP:** versión que adicionalmente consulta un servidor MCP externo (desarrollado en Python) que expone los criterios de clasificación y priorización como fuente de contexto consultable por protocolo estándar.

Ambas versiones procesan correctamente los cuatro casos de prueba definidos en el enunciado.

2. Arquitectura general del flujo

El flujo se diseñó como una secuencia de pasos especializados, donde cada nodo cumple una única responsabilidad. La entrada se recibe por Webhook, el caso recorre las etapas de validación, enriquecimiento, clasificación, priorización, resumen, construcción de salida, respuesta y trazabilidad.

Una decisión condicional (nodo IF) separa el camino de los casos válidos del de los casos incompletos: los válidos se procesan en su totalidad y los inválidos se rechazan indicando exactamente qué información falta.

2.1 Nodos del flujo principal

Nodo	Tipo	Función
Webhook	Entrada	Recibe el oficio en formato JSON por POST.
Verificar campos JSON	Code (Python)	Valida la presencia de los campos críticos.
Si JSON está completo	IF	Bifurca entre caso válido e incompleto.
Enriquecimiento del caso	Code (Python)	Deriva case_type, jurisdiction, owner y razón de prioridad.
Clasificación tecnológica	Code (Python)	Asigna la plataforma recomendada por árbol de decisión.
Priorización	Code (Python)	Calcula prioridad por modelo de puntaje ponderado.

Nodo	Tipo	Función
Generar Resumen	Google Gemini	Agente LLM que redacta el resumen ejecutivo.
Integrar resumen a JSON	Merge + Code	Une el resumen del LLM con el caso completo.
Construir JSON Final	Code (Python)	Arma la salida estructurada y ordenada.
Timestamp	Edit Fields	Añade la marca temporal de ejecución.
Respuesta al solicitante	Code (Python)	Genera el acuse de recibo.
Log Trazabilidad	Code (Python)	Registra la bitácora de la ejecución.
Respuesta Incompleta	Code (Python)	Rama de rechazo: informa los campos faltantes.

3. Lógica y reglas de negocio

3.1 Validación de entrada

El flujo verifica la presencia de los once campos críticos definidos en el enunciado (`case_id`, `requester_name`, `requester_email`, `area`, `process_name`, `document_type`, `description`, `current_solution`, `has_api_available`, `requires_human_approval`, `uses_sensitive_data`). Si falta alguno, el caso se rechaza y se devuelve la lista exacta de campos ausentes. Se trata explícitamente el valor booleano `false` como válido, evitando marcar como faltantes campos legítimamente falsos (por ejemplo `has_api_available`).

3.2 Clasificación tecnológica

La clasificación aplica un árbol de decisión evaluado en orden, de la opción más exigente a la más simple:

- **Appian:** apertura formal de caso, aprobación humana y trazabilidad legal (requiere aprobación humana y datos sensibles o urgencia regulatoria).
- **Microservicio:** lógica o cálculo especializado y complejo, identificado en la descripción del caso.
- **RPA Selectivo:** el caso opera sobre RPA y no existe API disponible (interfaz legacy sin alternativa).
- **n8n:** extracción, validación, normalización, enrute y notificación orquestables vía API.
- **Power Platform:** apoyo operativo simple de baja complejidad (caso por defecto).

El orden de evaluación es deliberado: garantiza que un caso crítico no termine clasificado en una categoría más simple. Durante las pruebas se detectó y corrigió un caso límite en el que el alto volumen desviaba indebidamente a Microservicio; la regla se ajustó para reservar Microservicio únicamente a la lógica diferenciadora, no al volumen.

3.3 Priorización

La prioridad se calcula con un modelo de puntaje ponderado, donde cada factor de riesgo o urgencia aporta puntos: urgencia regulatoria, sensibilidad de datos, aprobación humana, volumen mensual, tiempo manual, dependencia de RPA, ausencia de API y tipo de medida judicial. El puntaje total se traduce a Alta, Media o Baja. Este enfoque es transparente (se registran el puntaje y los factores) y fácil de ajustar, frente a una condición simple.

4. Manejo de errores

El manejo de errores opera en dos niveles:

1. **Errores de negocio:** la validación de campos críticos rechaza de forma controlada los casos incompletos, devolviendo un mensaje claro al solicitante.
2. **Errores técnicos:** se implementó un workflow independiente con un nodo Error Trigger, vinculado al flujo principal mediante la configuración Error Workflow. Ante cualquier fallo inesperado de un nodo, n8n dispara automáticamente este flujo y registra el evento. Adicionalmente, el nodo del agente LLM tiene activado el reintento automático (Retry On Fail) para tolerar indisponibilidades momentáneas del servicio.

5. Integración MCP (valor agregado)

Como valor agregado se desarrolló un servidor MCP (Model Context Protocol) en Python que expone los criterios de decisión del negocio como herramientas consultables. El servidor publica tres herramientas:

- **get_classification_criteria:** devuelve las reglas de clasificación tecnológica.
- **get_priority_rules:** devuelve los factores y pesos de priorización.
- **get_response_template:** devuelve la estructura del acuse de recibo.

El concepto que demuestra esta integración es la externalización de las reglas de negocio: en lugar de mantener los criterios fijos dentro del flujo, viven en una fuente externa que se consulta por un protocolo estándar. Así, las reglas pueden actualizarse sin modificar el flujo, y podrían ser compartidas por múltiples consumidores.

5.1 Topología de la integración

Dado que el flujo se ejecuta en n8n Cloud, este no puede alcanzar un servidor en localhost. Por ello, el servidor MCP se ejecuta localmente y se expone a internet mediante un túnel ngrok, que entrega una URL pública consumida por el nodo MCP Client de n8n a través del transporte SSE. En la versión con MCP, el flujo consulta los criterios en tiempo real y los adjunta al resultado final como evidencia de la fuente de decisión utilizada.

6. Ejemplo de entrada y salida

6.1 Entrada (caso completo)

```
{
  "case_id": "EMB-2026-00421",
  "requester_name": "Carolina Torres",
  "requester_email": "carolina.torres@empresa.com",
  "area": "Cumplimiento y Operaciones",
  "process_name": "Embargo judicial sobre cuenta de cliente",
  "document_type": "oficio_embargo",
  "description": "Oficio judicial para aplicar embargo preventivo.",
  "channel": "correo",
  "monthly_volume": 320,
  "current_solution": "rpa",
  "has_api_available": false,
  "requires_human_approval": true,
  "uses_sensitive_data": true,
  "regulatory_urgency": true,
  "estimated_manual_time_minutes": 25,
  "attachments": ["oficio_embargo.pdf", "anexos_judiciales.pdf"]
}
```

6.2 Salida estructurada

```
{
  "case_id": "EMB-2026-00421",
  "status": "received",
  "valid": true,
  "recommended_platform": "Appian",
  "priority": "Alta",
  "case_type": "embargo",
  "jurisdiction": "judicial",
  "recommended_owner": "legal",
  "summary": "Oficio de embargo judicial con prioridad Alta. Se recomienda Appian por aprobación humana y trazabilidad legal...",
  "priority_score": 13,
  "priority_factors": ["urgencia regulatoria", "datos sensibles", ...],
  "missing_fields": [],
  "next_step": "Registrar caso, validar anexos y escalar a revisión humana",
  "timestamp": "2026-06-11T21:10:17-05:00",
}
```

```
"acknowledgement_message": "Estimado(a) solicitante, ...",
"trace_log": { ... }
}
```

7. Casos de prueba verificados

Los cuatro casos definidos en el enunciado fueron ejecutados y validados:

- **Caso 1:** solicitud completa. Se clasifica, prioriza y responde correctamente.
- **Caso 2:** solicitud incompleta (faltan requester_email y document_type). El flujo rechaza el caso e informa los campos faltantes.
- **Caso 3:** caso crítico regulatorio. Se recomienda Appian con prioridad Alta.
- **Caso 4:** integración simple (sin aprobación humana, con API). Se recomienda n8n.

8. Notas de diseño

8.1 Supuestos

- **Entrada:** La entrada se recibe por Webhook en formato JSON, opción permitida explícitamente por el enunciado. Por ejecutarse en n8n Cloud, no se emplearon rutas de archivo locales.
- **Formato:** Se asume que la extracción del PDF escaneado a JSON estructurado ocurre en un paso previo (OCR/agente). El flujo recibe el oficio ya estructurado.
- **Ciclo de vida:** Las carpetas inbox/processed/errors/logs del enunciado se representan como estados lógicos del caso (campo status, rama de error y bitácora) en lugar de movimientos físicos de archivos.
- **MCP:** El servidor MCP se expone vía ngrok; la URL pública es temporal. Para reproducir la integración, el evaluador debe levantar el servidor y el túnel, o usar la evidencia adjunta.
- **LLM:** El agente de resumen usa Google Gemini con clave propia. La credencial no se incluye en el export; debe configurarse una clave propia para reproducir.

8.2 Riesgos

- **Dependencia externa:** la versión con MCP depende de que el servidor Python y el túnel estén activos. La versión base es autónoma y no tiene esta dependencia.
- **URL temporal:** la URL de ngrok cambia al reiniciar en el plan gratuito; debe actualizarse en el nodo MCP Client.
- **Disponibilidad del LLM:** el servicio LLM puede presentar indisponibilidades momentáneas, mitigadas con reintento automático.
- **Vínculo de errores:** al importar los flujos, el vínculo del Error Workflow debe re-asignarse en Settings del flujo principal.

8.3 Posibles mejoras

- **MCP como cerebro de decisión:** incorporar un AI Agent que consuma directamente las herramientas del MCP para decidir la clasificación de forma dinámica, en lugar de reglas locales.
- **Extracción de PDF:** añadir un paso real de OCR/extracción para procesar PDFs escaneados de extremo a extremo.
- **Persistencia del log:** enviar la bitácora a un destino persistente (Google Sheets o base de datos) para auditoría histórica.
- **Limpieza del resumen:** instruir al LLM para devolver texto plano sin formato markdown, dejando el resumen completamente limpio.
- **Despliegue del MCP:** desplegar el servidor MCP en un hosting permanente para obtener una URL estable.

9. Contenido de la entrega

- **1:** Export JSON del flujo base (versión autónoma).
- **2:** Export JSON del flujo con MCP.
- **3:** Export JSON del workflow de manejo de errores.
- **4:** Código del servidor MCP en Python (`servidor_mcp.py`).
- **5:** Capturas de evidencia de la integración MCP en funcionamiento (servidor recibiendo peticiones, túnel activo, consulta desde n8n y diagrama del flujo).
- **6:** El presente documento de explicación técnica, ejemplo de entrada/salida y notas de diseño.